

```

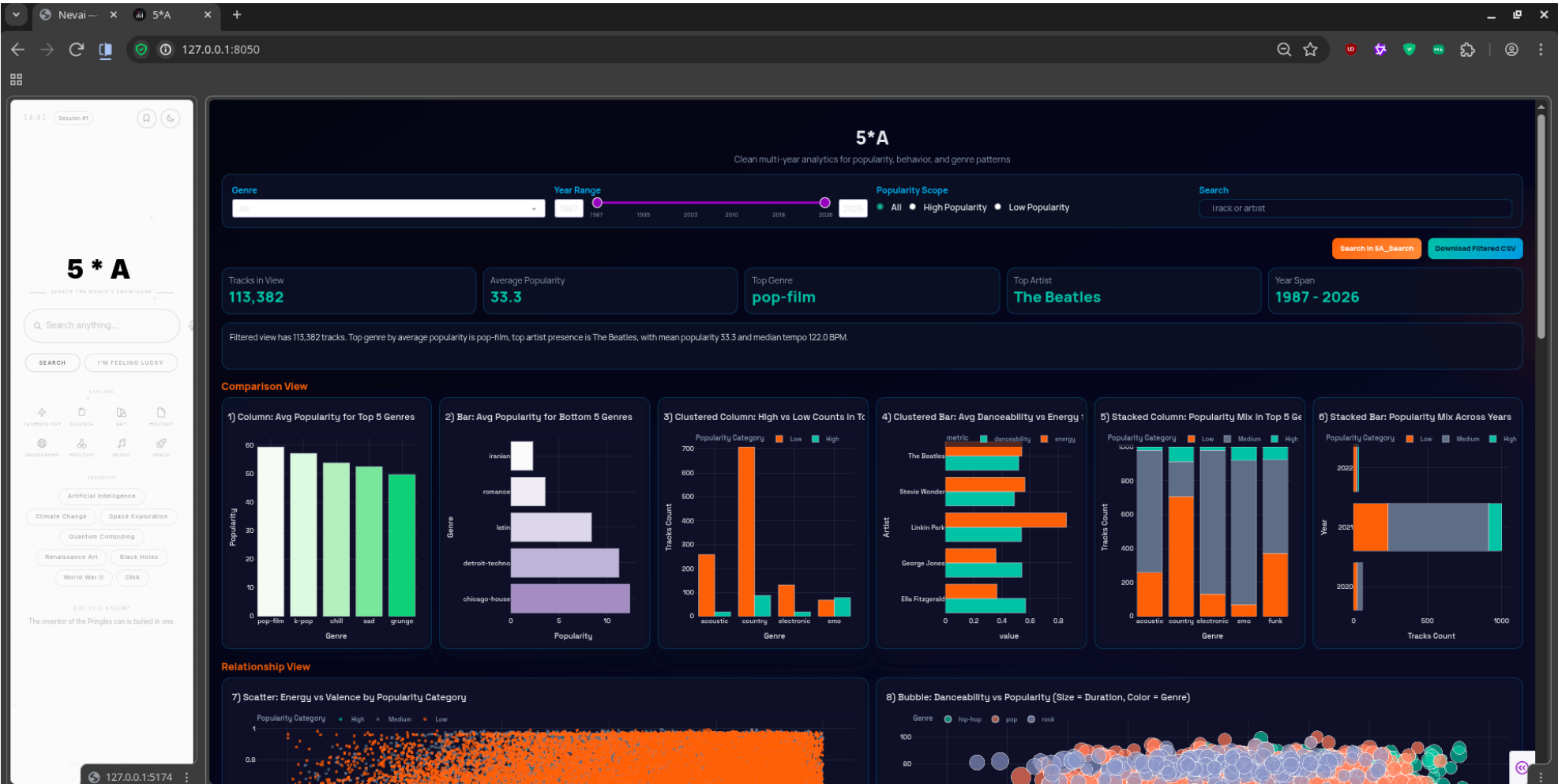
+
./run_all_apps.sh both
~/Downloads/DV_Project

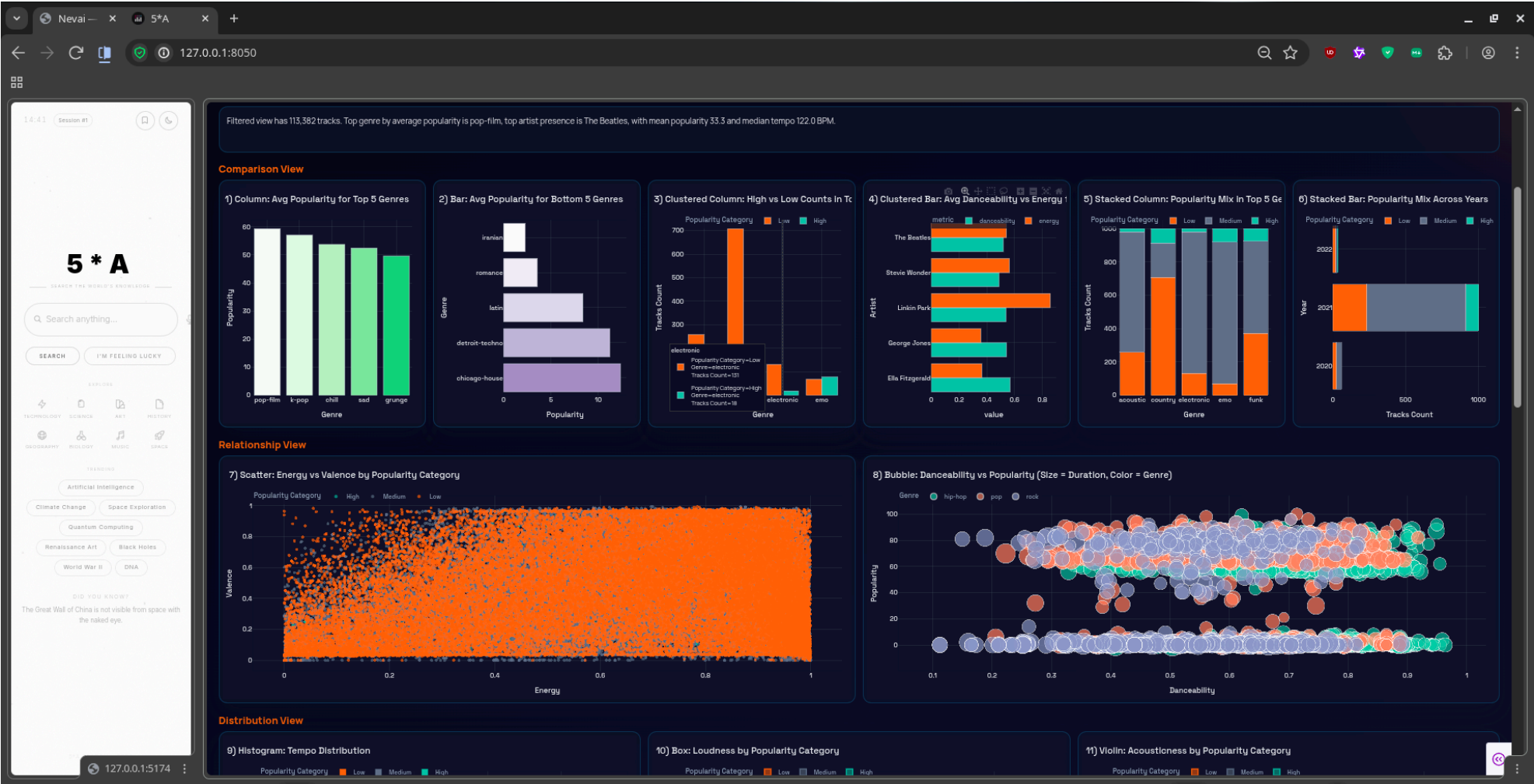
VITE v6.3.5  ready in 253 ms

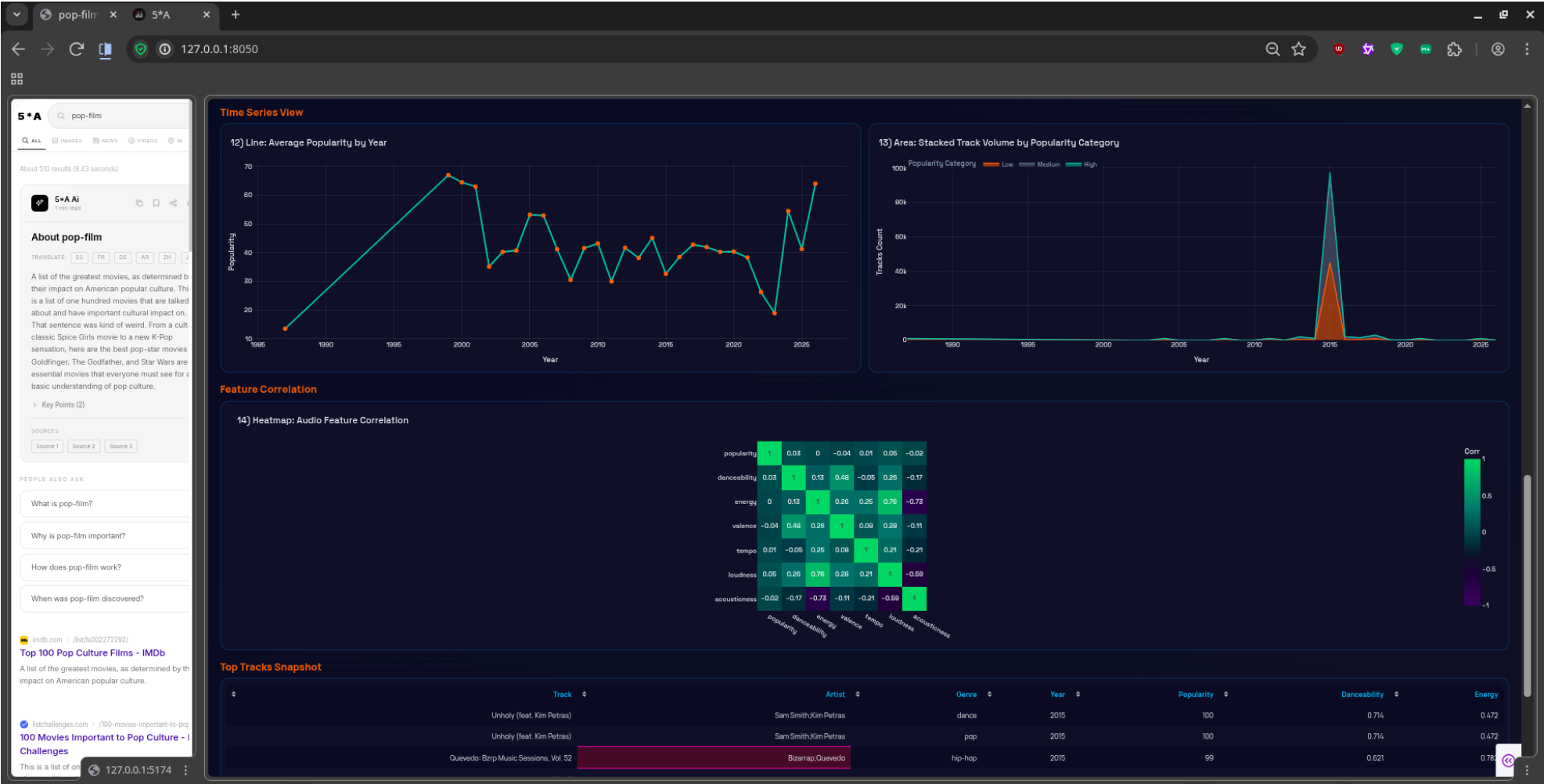
+ Local:   http://localhost:5174/
+ Network: http://192.168.1.11:5174/
[INFO] 5A_Search ready at http://127.0.0.1:5174
Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'app'
* Debug mode: on
[INFO] Spotify Dashboard ready at http://127.0.0.1:8050
[INFO]
[INFO]
[INFO] ✓ All Apps Running
[INFO]
[INFO] Spotify Dashboard: http://127.0.0.1:8050
[INFO] 5A_Search Engine: http://127.0.0.1:5174
[INFO]
[INFO]

```







pop-film

5*A

127.0.0.1:8050

5*A

pop-film

ALLIMAGESNEWSVIDEOSM

About 510 results (8.43 seconds)

5*A AI

1 min read

About pop-film

TRANSLATE:ESFRDEARZH

A list of the greatest movies, as determined by their impact on American popular culture. This is a list of one hundred movies that are talked about and have important cultural impact on. That sentence was kind of weird. From a cult classic Spice Girls movie to a new K-Pop sensation, here are the best pop-star movies Goldfinger, The Godfather, and Star Wars are essential movies that everyone must see for a basic understanding of pop culture.

> Key Points (2)

SOURCES

Source 1Source 2Source 3

PEOPLE ALSO ASK

What is pop-film?

Why is pop-film important?

How does pop-film work?

When was pop-film discovered?

imdb.com / list/500272292

Top 100 Pop Culture Films - IMDb

A list of the greatest movies, as determined by their impact on American popular culture.

listchallenges.com / 100-movies-important-to-pop

100 Movies Important to Pop Culture - I Challenges

This is a list of on

20

10

1985

1990

1995

2000

2005

2010

2015

2020

2025

Year

20k

0

1990

1995

2000

2005

2010

2015

2020

2025

Year

Feature Correlation

14) Heatmap: Audio Feature Correlation

popularity

danceability

energy

valence

tempo

loudness

acousticness

popularity

danceability

energy

valence

tempo

loudness

acousticness

Corr

1

0.5

0

-0.5

-1

Top Tracks Snapshot

	Track	Artist	Genre	Year	Popularity	Danceability	Energy
1	Unholy (feat. Kim Petras)	Sam Smith, Kim Petras	dance	2015	100	0.714	0.472
2	Unholy (feat. Kim Petras)	Sam Smith, Kim Petras	pop	2015	100	0.714	0.472
3	Quevedo: Bzrp Music Sessions, Vol. 52	Bizarrap, Quevedo	hip-hop	2015	99	0.621	0.782
4	I'm Good (Blue)	David Guetta, Bebe Rexha	dance	2015	98	0.561	0.965
5	I'm Good (Blue)	David Guetta, Bebe Rexha	edm	2015	98	0.561	0.965
6	I'm Good (Blue)	David Guetta, Bebe Rexha	pop	2015	98	0.561	0.965
7	La Bachata	Manuel Turizo	latin	2015	98	0.835	0.679
8	La Bachata	Manuel Turizo	latino	2015	98	0.835	0.679
9	La Bachata	Manuel Turizo	reggae	2015	98	0.835	0.679
10	La Bachata	Manuel Turizo	reggaeton	2015	98	0.835	0.679
11	Titi Me Preguntó	Bad Bunny	latin	2015	97	0.65	0.715
12	Titi Me Preguntó	Bad Bunny	latino	2015	97	0.65	0.715

Lady Gaga

5*A

127.0.0.1:5174?q=Lady%20Gaga

5*A

Lady Gaga

ALLIMAGESNEWSVIDEOSMAPS

About 590 results (1.92 seconds)

5*A AI

2 min read

About Lady Gaga

TRANSLATE:ESFRDEARZHJA

Stefani Joanne Angelina Germanotta, known professionally as Lady Gaga, is an American singer, songwriter, and actress. ❤️ Milla Miranda, 36 Lady Gaga is an American singer-songwriter and performance artist, known for her flamboyant costumes, provocative lyrics, and strong vocal talents. Grammy Museum for the 'Lady Gaga in Harlem Live: One Night Only' Premiere 📺 Watch it on my YouTube channel @ 4pm PT on Xmas Eve. Lady Gaga, born Stefani Joanne Angelina Germanotta, is an American songwriter, singer, actress, philanthropist, dancer and fashion designer.

> Key Points (5)

SOURCES

Source 1Source 2Source 3

PEOPLE ALSO ASK

What is Lady Gaga?

Why is Lady Gaga important?

How does Lady Gaga work?

When was Lady Gaga discovered?

en.wikipedia.org / wiki/Lady_Gaga

Lady Gaga - Wikipedia

Stefani Joanne Angelina Germanotta was born on March 28, 1986, at Lenox Hill Hospital in the neighborhood of Lenox Hill, Manhattan, New York City, United States. [1] Her parents are Cynthia Louise (née

Lady Gaga

AMERICAN SINGER, SONGWRITER AND ACTRESS (BORN 1986)

Stefani Joanne Angelina Germanotta, known professionally as Lady Gaga, is an American singer, songwriter, and actress. Known for her image reinventions and versatility across the entertainment industry, she is an influential figure in popular music. With estimated sales of 124 million records, she i...

WIKIPEDIA SOURCE

RELATED TOPICS

Lady Gaga Category

Artists with the most number-ones on the U.S. Dance Club Songs chart ~...

Forbes list of highest-earning musicians - The highest-earning musica...

Honoric nicknames in popular music - When describing popular music a...

Anti-fracking movement

Convent of the Sacred Heart (NYC) alumni

5*A

Clean multi-year analytics for popularity, behavior, and genre patterns

Genre

Year Range

Popularity Scope

Search

Search in 5A_Search

Download Filtered CSV

Tracks in View

80

Average Popularity

14.7

Top Genre

german

Top Artist

Lady Gaga

Year Span

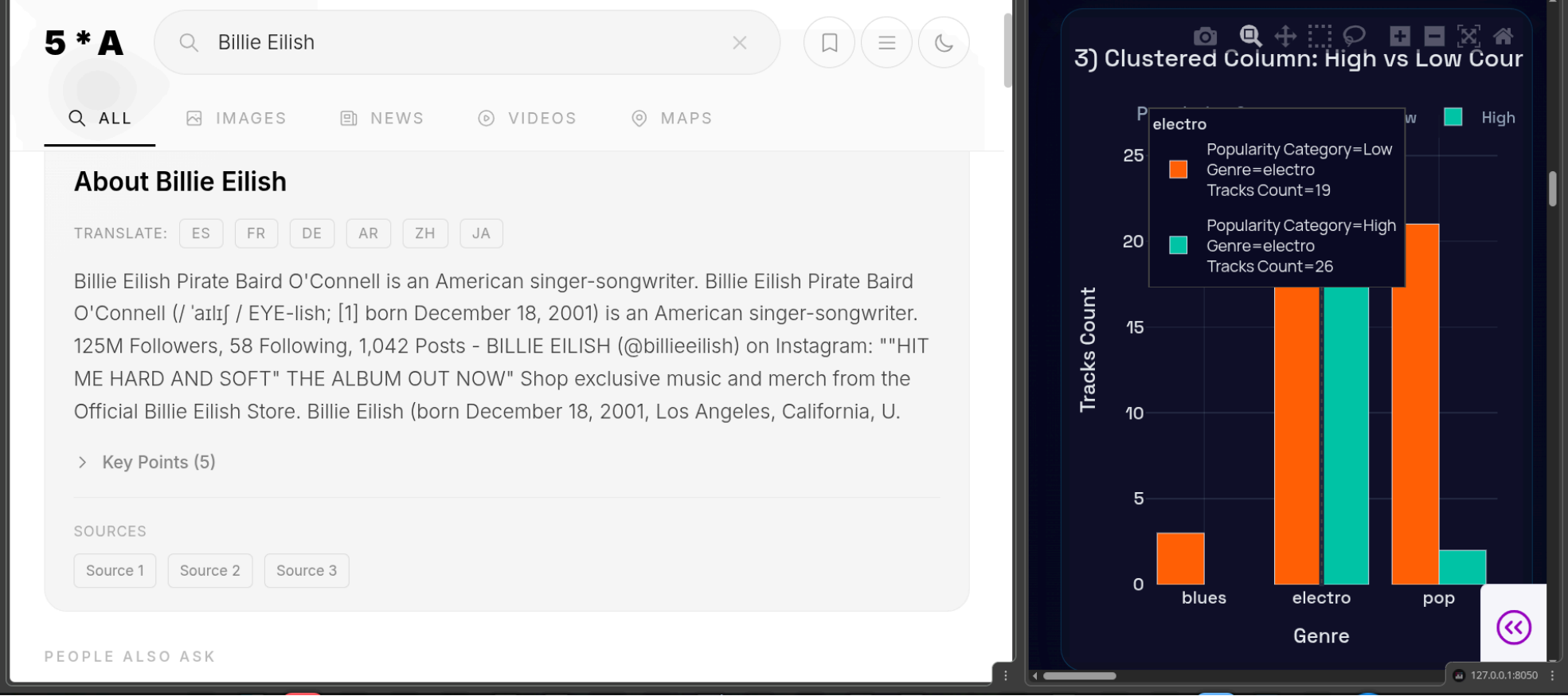
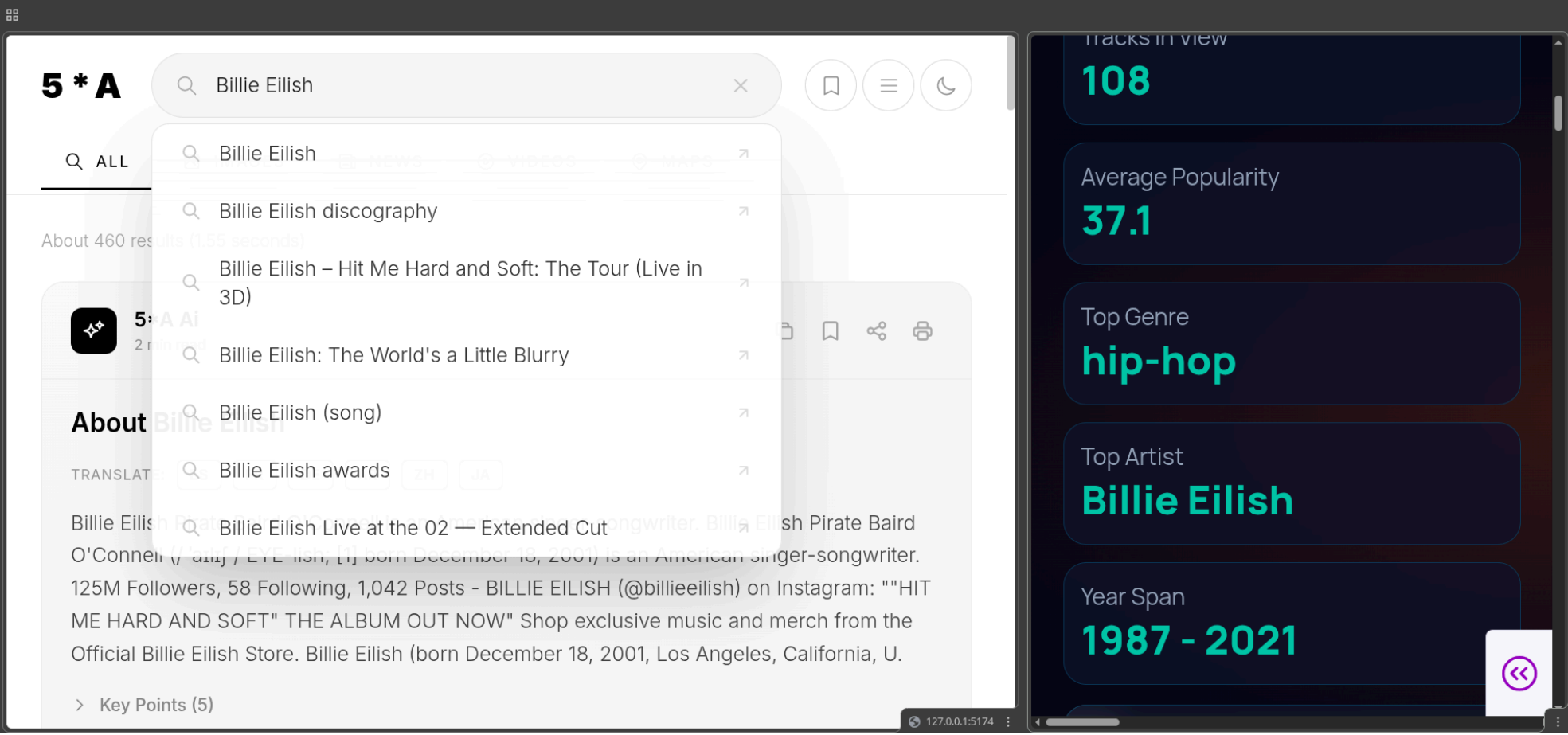
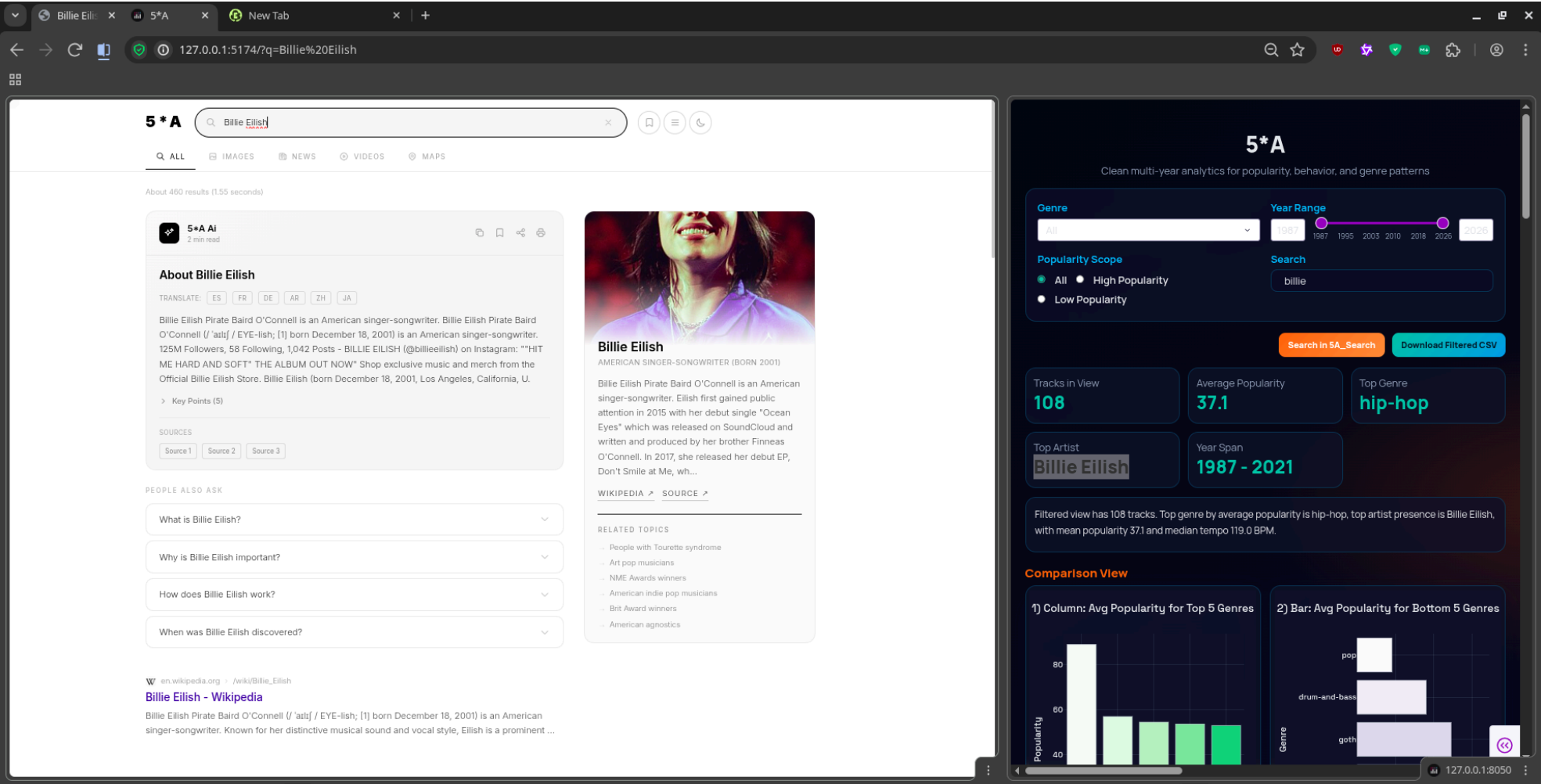
1987 - 2018

Filtered view has 80 tracks. Top genre by average popularity is german, top artist presence is Lady Gaga, with mean popularity 14.7 and median tempo 120.0 BPM.

Comparison View

1) Column: Avg Popularity for Top 5 Genres

2) Bar: Avg Popularity for Bottom 5 Genres



Comprehensive Documentation: Nevai Search - Spotify Data Pipeline

Project Team

Order ID	Number	Name	Role in Project	Responsibilities
1	931230015	Ahmed Abobakr Hanafi	Project Manager / System Architect	Overall project supervision, architectural decisions, code review, team coordination
2	931230187	Ali Ez Alyan	Backend Lead Developer	API development, database management, security implementation, performance optimization
3	931230030	Ahmed Sabry Hamza	Frontend Developer	React/Vue UI development, user experience design, animations, dark/light theme implementation
4	931230037	Ahmed AbdulNasser Sayed	API Integration Specialist	Integrating 20+ search sources, data processing, error handling, response optimization
5	931230046	Ahmed Mohamed Mohamed	QA Engineer & Documentation	Testing, code review, project documentation, quality assurance

Project Goals and Technological Rationale

1. The Overarching Goal of the Project

The primary objective of this project is to engineer a unified, full-stack software ecosystem that demonstrates advanced proficiency in data engineering, complex API orchestration, artificial intelligence integration, and user interface design. Rather than building isolated scripts, the team constructed two interconnected, production-grade applications under a single architecture:

- Nevai Search Engine:** A privacy-first, AI-enhanced meta-search engine designed to consolidate fragmented internet data into a single, minimalist interface.
- 5*A Task Management System:** A comprehensive productivity suite integrated with the search engine to demonstrate full-stack backend (FastAPI) and frontend (Vanilla JS/React) capabilities.

The underlying academic and professional goal is to solve the modern problem of information fragmentation and data privacy. By building Nevai, the project proves that it is possible to aggregate massive amounts of data from over 20 disparate sources, parse it, and present it to the user without relying on third-party data harvesting, all while using local AI to synthesize the information.

2. Why Nevai Search?

Traditional search engines are increasingly monopolized, ad-supported, and reliant on capturing user data to function. Nevai was designed to provide an alternative paradigm based on three core pillars:

A. Aggregation of Fragmented Data

Users currently jump between Wikipedia for facts, Reddit for discussions, GitHub for code, and OpenMeteo for weather. Nevai centralizes this by integrating over 20 distinct APIs (including DuckDuckGo, SearXNG, Wikimedia projects, arXiv, and RestCountries) into a single query endpoint. The goal is to reduce context-switching for the user by providing tabbed results (Web, News, Images, Maps) in one interface.

B. Zero-Friction Privacy

Most meta-search engines or API aggregators require end-users to provide their own API keys (e.g., Google API keys, Spotify Client IDs). Nevai is explicitly designed to require **zero API keys from the end-user**. The backend handles the complex routing, rate limiting, and API credential management transparently.

C. Minimalist and Functional UI

Modern search result pages are cluttered with advertisements, SEO spam, and extraneous UI elements. Nevai employs a Zara-inspired, strict black-and-white minimalist design philosophy (using typography like Cormorant Garamond and Space Grotesk). The goal is to prioritize information density and readability over clickbait aesthetics.

3. Why AI, and Why Local AI?

Artificial Intelligence is integrated into Nevai not as a gimmick, but as a necessary middleware layer to make sense of aggregated data. Raw API responses from 20 different sources are structurally inconsistent; AI is used to normalize and synthesize this data for the user.

A. Context-Aware Synthesis

When a user searches for a complex topic (e.g., "Egypt" or "Holy Quran"), they receive long Wikipedia extracts, news articles, and map data. The AI component generates concise summary cards and extracts exactly "5 Key Points" from these disparate sources. Furthermore, the AI maintains conversational context, allowing users to ask follow-up questions (e.g., "What is the population there?") without restating the original search query.

B. The Critical Choice: Local Execution via WebAssembly (web-11m)

The most defining technical decision in this project is the rejection of cloud-based AI APIs (like OpenAI or Anthropic) in favor of running Large Language Models (LLMs) entirely locally in the user's browser using WebAssembly through the web-11m library.

This decision was made for the following reasons:

- Absolute Privacy:** Sending user search queries to a third-party AI API completely violates the privacy-first philosophy of the project. By running the model locally, zero search data, prompts, or metadata ever leave the user's device.
- No API Costs or Rate Limits:** Cloud AI APIs charge per token and implement strict rate limits. A local LLM bypasses these constraints entirely, allowing unlimited AI interactions without backend infrastructure costs.
- Offline Capability Planning:** By decoupling the AI from the server, the application lays the groundwork for Progressive Web App (PWA) offline capabilities in future iterations.

C. Demonstrating Data Pipeline Mastery

The AI integration also serves to highlight the team's data engineering capabilities. As demonstrated in the Spotify Data Pipeline (five_a_data_prep.ipynb), the team has proven its ability to design complex, fault-tolerant data extraction systems (e.g., the 6-tier year extraction cascade using Spotify and MusicBrainz APIs). This same architectural philosophy—using multiple fallback mechanisms, local caching, and rigorous data sanitization—is applied to the AI and search APIs in Nevai to ensure the application never crashes or returns empty results.

The primary purpose of this application module is to serve as the robust data ingestion, cleaning, and preprocessing pipeline for the Spotify Trends Dashboard component of the Nevai Search application.

The high-level architecture follows a sequential Extract, Transform, Load (ETL) pattern. The system ingests a raw, unstructured Spotify tracks dataset (dataset.csv). It processes this raw data through a rigorous, multi-tiered pipeline. First, it standardizes the schema and coerces data types. Second, it executes a sophisticated year-extraction algorithm that relies on a cascading series of fallback mechanisms, ranging from local caching to external API queries (Spotify and MusicBrainz) and concluding with statistical imputation. Third, it enforces strict data quality rules, removing null values, fake genres, and out-of-bounds numerical data. Finally, it engineers a new categorical feature (Popularity_Category) and exports the sanitized, analysis-ready dataset (spotify_cleaned.csv) to be consumed by the interactive Dash visualization layer (app.py).

2. Tech Stack and Prerequisites

Programming Languages and Core Libraries

- Python:** Primary programming language for data processing and pipeline logic.
- Pandas (>=2.2.2):** Utilized for data manipulation, DataFrame transformations, schema mapping, and cleaning.
- NumPy (>=2.1.1):** Utilized for numerical operations, mathematical coercions, and explicit missing value handling (np.nan).

Visualization and Environment

- Dash (>=2.18.2):** Web framework for the downstream interactive dashboard that consumes the cleaned data.
- Plotly (>=5.24.1):** Graphing library used by Dash to render interactive visualizations.
- JupyterLab (>=4.2.5):** Interactive development environment used to author and execute the data preparation notebook.

External APIs

- Spotify Web API:** Used for high-confidence release year extraction via album metadata.
- MusicBrainz API:** Used as a credential-free fallback for release year extraction via recording metadata.

Required Environment Variables

- SPOTIFY_CLIENT_ID : Spotify API client ID (optional; skips Spotify API tier if unset).
- SPOTIFY_CLIENT_SECRET : Spotify API client secret (optional; skips Spotify API tier if unset).
- MB_MAX_LOOKUPS : Integer defining the maximum number of MusicBrainz API requests to execute (default: 120).

3. Setup and Execution

Installation

1. Navigate to the project directory (`spotify_trends_dashboard`).
2. Create and activate a virtual environment (recommended).
3. Install the required Python dependencies using the provided requirements file:

```
pip install -r requirements.txt
```

Execution

1. Ensure the raw dataset (e.g., `dataset.csv`) is located in the project root or its immediate parent directory.
2. Launch JupyterLab:

```
jupyter lab
```

3. Open and execute the `five_a_data_prep.ipynb` notebook sequentially from top to bottom. This process will generate `spotify_cleaned.csv` and update the local metadata cache.
4. Once the data pipeline completes successfully, launch the visualization dashboard from the terminal:

```
python app.py
```

4. Application Flow

The end-to-end data flow follows a strict, sequential process designed to maximize data fidelity:

1. **Ingestion:** The notebook dynamically searches predefined candidate paths for a raw CSV file. It loads the data (initially 114,000 rows by 21 columns) into a Pandas DataFrame and strips leading/trailing whitespace from all column headers.
2. **Schema Standardization:** A predefined mapping dictionary matches raw column names to a standardized 16-column schema. Numeric columns are coerced using `pd.to_numeric(..., errors="coerce")` to handle malformed strings gracefully.
3. **Year Extraction (6-Tier Cascade):**
 - *Tier 1 (Dataset):* Checks for existing `release_date` or `year` columns natively present in the file.
 - *Tier 2 (Local Cache):* Maps missing years using a pre-existing `spotify_track_metadata_years.csv` file.
 - *Tier 3 (Spotify API):* If environment credentials are present, queries the Spotify API in batches of 50 to fetch album release dates.
 - *Tier 4 (MusicBrainz API):* For remaining nulls, queries MusicBrainz by track name and normalized artist name.
 - *Tier 5 (Imputation):* Fills remaining nulls with the median year of the track's artist, then the median year of the track's genre.
 - *Tier 6 (Global Fallback):* Any stubborn missing values are filled with the global median year (clipped between 2000 and 2023). The year is rounded to an integer, and rows outside 1900-2035 are dropped.
4. **Data Cleaning:**
 - Drops rows with nulls in critical columns (`track_name` , `artists` , `popularity` , `danceability` , etc.).
 - Filters out fake genres (e.g., "unknown", "n/a", "nan", "none").
 - Clips numerical features to valid Spotify boundaries (popularity to 0-100, danceability to 0-1, etc.).
 - Converts durations recorded in seconds (values under 2000ms) to milliseconds, then filters to keep only tracks between 30,000ms and 900,000ms.
5. **Feature Engineering:** Creates an ordered categorical column `Popularity_Category` segmented into "Low" (0-29), "Medium" (30-69), and "High" (70-100).
6. **Deduplication:** Drops exact duplicates based on the composite key of `track_id` , `artists` , `track_name` , and `track_genre` .
7. **Export:** Saves the final 113,382-row DataFrame to `spotify_cleaned.csv` in both the project root and the `dataset_5a` subdirectory. Updates the local metadata cache.

5. Detailed

five_a_data_prep.ipynb

- **Purpose:** The core data preprocessing engine containing all ETL logic.
- **Contents:** Sequential Python code cells executing the pipeline described in Section 4.
- **Key Functions:**
 - `first_existing_column(df, options)` : Iterates through a list of alias names, returning the first match found in the DataFrame columns. Used for flexible schema standardization.
 - `coerce_year_int(value)` : Safely converts a value to an integer year. Returns `None` if the value is non-numeric or falls outside the strict 1900-2035 range.
 - `extract_year_from_release(value)` : Parses a string (e.g., "2017-05-12") to extract a float year using Pandas datetime parsing. Falls back to checking if the first 4 characters are digits.
 - `get_spotify_access_token(client_id, client_secret)` : Implements the OAuth2 Client Credentials flow. Encodes credentials in base64, POSTs to the Spotify accounts endpoint, and returns the Bearer token.

- `fetch_spotify_years(track_ids, token, batch_size=50, pause_seconds=0.12)` : Queries the `/v1/tracks` endpoint in batches, extracts the release date from the album object, and maps the Track ID to the coerced integer year.
- `_normalize_artist_name(text)` : Cleans artist strings by extracting only the primary artist (splits by `;`, `,`, or `&` and strips whitespace).
- `fetch_musicbrainz_year(track_name, artist_name)` : Constructs a MusicBrainz WS/2 API query string, makes a GET request with a custom User-Agent, and parses the first matching recording's release date.
- `fetch_musicbrainz_years(df_subset, max_lookups=120)` : Iterates over a DataFrame of missing tracks, calls the single-track fetch function, prints progress every 20 rows, and strictly enforces a 1.05-second sleep between requests.
- **Dependencies:** Standard library (`os`, `time`, `json`, `base64`, `pathlib`, `urllib`), `numpy`, and `pandas`.

requirements.txt

- **Purpose:** Defines the Python package dependencies for the pipeline and the downstream dashboard.
- **Contents:**
 - `dash>=2.18.2`
 - `pandas>=2.2.2`
 - `plotly>=5.24.1`
 - `numpy>=2.1.1`
 - `jupyterlab>=4.2.5`
- **Dependencies:** Required to run both the Jupyter notebook and the `app.py` dashboard.

spotify_cleaned.csv

- **Purpose:** The final, sanitized output dataset ready for analytical consumption.
- **Contents:** Contains 113,382 rows and 18 columns. Contains zero remaining null values. The `Popularity_Category` distribution is Medium (57,920), Low (49,994), and High (5,468). The year range spans from 1987 to 2026.
- **Dependencies:** Generated by `five_a_data_prep.ipynb`. Acts as the direct input dependency for `app.py`.

spotify_track_metadata_years.csv

- **Purpose:** A persistent local cache storing previously resolved release years for specific Spotify Track IDs.
- **Contents:** Two columns: `track_id` (string) and `Year` (integer). In the provided execution context, it contains 345 successfully resolved entries.
- **Dependencies:** Read by `five_a_data_prep.ipynb` during Tier 2 of the year extraction phase. Overwritten and updated at the end of the notebook execution. This file drastically reduces external API calls on subsequent pipeline runs.

6. Data Dictionary

The following schema defines the 18 columns present in the final `spotify_cleaned.csv` output:

Column Name	Data Type	Valid Range / Format	Description
track_id	String	22 alphanumeric characters	Unique Spotify identifier for the track.
artists	String	Variable length	Artist name(s), separated by semicolons if featuring multiple artists.
album_name	String	Variable length	The name of the album the track belongs to.
track_name	String	Variable length	The title of the track. Null values dropped.
track_genre	String	Variable length	Genre classification. Fake genres (e.g., "unknown") filtered out.
popularity	Float/Int	0 to 100	Metric calculating current track popularity. Clipped to strict bounds.
duration_ms	Float/Int	30,000 to 900,000	Track length in milliseconds. Values <2000 were multiplied by 1000 to convert seconds.
danceability	Float	0.0 to 1.0	Suitability for dancing based on tempo and rhythm. Clipped to bounds.
energy	Float	0.0 to 1.0	Perceptual measure of intensity and activity. Clipped to bounds.
valence	Float	0.0 to 1.0	Musical positiveness (sad vs. happy). Clipped to bounds.
tempo	Float	Variable (typically 50-200)	Estimated tempo in beats per minute (BPM).
loudness	Float	Variable (typically -60 to 0)	Overall loudness in decibels (dB).
acousticness	Float	0.0 to 1.0	Confidence measure of acoustic nature. Clipped to bounds.
speechiness	Float	0.0 to 1.0	Detects presence of spoken words.

Column Name	Data Type	Valid Range / Format	Description
instrumentalness	Float	0.0 to 1.0	Predicts whether a track contains no vocals.
liveness	Float	0.0 to 1.0	Detects presence of an audience in the recording.
Year	Integer	1900 to 2035	Release year derived via the multi-tier extraction pipeline.
Popularity_Category	Categorical	"Low", "Medium", "High"	Engineered feature. Low (0-29), Medium (30-69), High (70-100).

7. External Services and APIs

Spotify Web API

- Purpose:** Primary source for high-confidence release year extraction.
- Authentication:** OAuth2 Client Credentials flow. Requires `SPOTIFY_CLIENT_ID` and `SPOTIFY_CLIENT_SECRET` environment variables. Credentials are base64 encoded and sent to `https://accounts.spotify.com/api/token`.
- Endpoint Used:** GET `https://api.spotify.com/v1/tracks?ids={comma_separated_ids}`
- Rate Limiting:** Handled proactively via a `pause_seconds=0.12` delay between batch requests and a strict batch size limit of 50 track IDs per request.
- Fallback Mechanism:** Entirely skipped if environment variables are not set, printing a status message: "Spotify credentials not set; skipping Spotify API enrichment." If an HTTP or URL error occurs during a batch fetch, it sleeps for 1.2 seconds and continues to the next batch.

5*A API

- Purpose:** Secondary, credential-free fallback source for release year extraction.
- Authentication:** None required. Mandates a descriptive `User-Agent` header (`DVProjectBot/1.0 (academic-project)`) per MusicBrainz terms of service.
- Endpoint Used:** GET `https://musicbrainz.org/ws/2/recording/?query={url_encoded_query}&fmt=json&limit=1`
- Rate Limiting:** Strictly enforced. The implementation applies a hard `time.sleep(1.05)` delay between every single request to comply with the "1 request per second" guideline.
- Query Logic:** Constructs a query combining the exact track name and normalized primary artist name (e.g., `recording:"Track Name" AND artist:"Artist"`).
- Fallback Mechanism:** Limits total lookups via the `MB_MAX_LOOKUPS` environment variable. Fails gracefully on `HTTPError` or `URLError` by returning `np.nan` rather than halting the pipeline.

8. Configuration and Caching

Environment Variables

The pipeline relies on environment variables for dynamic configuration without requiring code changes:

- `SPOTIFY_CLIENT_ID` / `SPOTIFY_CLIENT_SECRET` : Boolean gate controlling access to Tier 3 of the year extraction pipeline.
- `MB_MAX_LOOKUPS` : Controls the maximum volume of requests sent to the MusicBrainz API, allowing developers to adjust runtime based on network constraints or strict compliance requirements.

File Search Configuration

The ingestion step avoids brittle hardcoded file paths. Instead, it utilizes a prioritized candidate list:

```
candidates = [
    PROJECT_DIR / "spotify_tracks.csv",
    PROJECT_DIR / "spotify_tracks_dataset.csv",
    PROJECT_DIR / "spotify.csv",
    PROJECT_DIR / "dataset.csv",
    PROJECT_DIR.parent / "dataset.csv",
]
```

It iterates through this list and assigns the first existing file to `raw_path`, ensuring flexibility across different dataset naming conventions.

Local Metadata Caching

To prevent redundant external API calls, which are slow and strictly rate-limited, the system implements a persistent local cache via `spotify_track_metadata_years.csv`.

- Read Strategy:** At the start of the year extraction phase, the notebook loads this CSV into a dictionary mapping `track_id` to `Year`. It immediately applies this dictionary to fill any rows currently missing a year.

- **Write Strategy:** After resolving years via the Spotify API or MusicBrainz, the newly resolved IDs are merged into the in-memory dictionary. At the end of the execution, the entire merged dictionary is exported back to the same CSV file, overwriting previous contents. This ensures that subsequent runs of the notebook will instantly resolve previously processed tracks without requiring any network calls.